Campus Emergency Supply Tracker (QuickRelief)

Objective: Build a lightweight, functional REST API backend in 2 hours to track and distribute campus emergency medical kits and food packets during seasonal flash floods/campus emergencies.

---

1. Challenge Overview & Technical Constraints

- **Time Limit:** 2 Hours (Beginner-Friendly scope)
- **Language & Runtime:** Java 17+
- **Framework & Build Tool:** Spring Boot 3.x, Apache Maven
- **Persistence Layer:** Spring JdbcTemplate or raw JDBC (H2 in-memory, MySQL, or PostgreSQL)
- **Strict Constraint:** No ORM/JPA frameworks (e.g., Hibernate or Spring Data JPA) allowed. Use explicit SQL queries.

---

2. Database Schema

The database consists of only two simplified tables:

1. **supplies:** Stores current inventory items (`id`, `item_name`, `quantity`)
2. **requests:** Tracks supply allocations (`id`, `item_name`, `quantity`, `status`, `requested_by`)

---

3. Core User Stories (Endpoints to Build)

- **POST /api/supplies:** Add new emergency inventory items or restock existing supply levels.
- **GET /api/supplies:** View all available inventory using a custom JdbcTemplate `RowMapper`.
- **POST /api/requests:** Claim or allocate an item with transactional validation (verify stock availability, deduct remaining quantity, and record request details).

---

4. Suggested 2-Hour Time Budget

- **00:00 - 00:20 | Setup & Configuration:** Set up Maven `pom.xml`, database properties in `application.properties`, and initialize database tables via `schema.sql`.
- **00:20 - 01:00 | Persistence Layer:** Write domain models and JdbcTemplate repositories with raw SQL queries and RowMappers.
- **01:00 - 01:30 | Business & API Layer:** Build the Service layer using `@Transactional` and implement REST controllers.
- **01:30 - 02:00 | Testing & Verification:** Perform API testing via Postman/curl and verify edge cases (e.g., rejecting requests exceeding available stock).

---

5. Definition of Done & Success Criteria

- All 3 REST endpoints function as requested and successfully process valid JSON payloads.
- All database interactions use raw SQL queries via JdbcTemplate with zero JPA/ORM dependencies.
- The supply request pipeline safely verifies and updates inventory within an explicit transactional boundary.